

Simulation and Defense of Web Attacks

Project Report

Course: Cyber Security – University of Messina

Instructor: Prof. Francesco Longo

Student: Rana Sanjideh



June 2, 2025

1. Introduction

This project uses defense mechanisms based on the security concepts covered in your course to mimic two prevalent and significant online application attacks: cross-site request forgery (CSRF) and brute-force login attacks.

Understanding how these attacks take advantage of vulnerable online applications and how appropriate mitigation strategies, such as rate limitation, CAPTCHA, CSRF tokens, and SameSite cookies, safeguard users and systems is the main objective.

2. Objectives

- Simulate a brute-force login attack on a vulnerable login system.
- Implement basic defenses: CAPTCHA and request rate limiting.
- Simulate a CSRF attack that attempts to change user data without consent.
- Implement CSRF protection using tokens and SameSite cookies.
- Evaluate the effectiveness of defenses.

3. Attack Simulation Scenarios

3.1. Brute-Force Login Attack

A Python script automatically submits login attempts with different credentials. Without rate limiting and CAPTCHA, the script can successfully guess the correct password.

3.2. CSRF Attack

An attacker page simulates a malicious site that auto-submits a form to change the victim's email. If the victim is logged in, their email is changed silently without consent.

4. Defense Mechanisms

- **CAPTCHA:** Blocks automated bots by requiring a human answer.
- **Rate Limiting:** Restricts the number of login attempts per minute.
- **CSRF Tokens:** Prevent unauthorized form submissions using Flask-WTF.
- **SameSite Cookies:** Browser-enforced protection against cross-site requests.

5. Implementation Details

- Backend: Python with Flask
- Templates: (login.html, dashboard.html, change_email.html, attacker.html)
- Security Libraries: Flask-Limiter, Flask-WTF, hashlib

6. Testing & Results

Test Case	Condition	Result
Brute-force login (no defenses)	No CAPTCHA or rate limiting	Success – Password guessed in few attempts
Brute-force login (with CAPTCHA)	CAPTCHA added to login form	Blocked – Automation disrupted by human challenge
Brute-force login (with rate limiting)	5 attempts per minute allowed	Blocked after 5 incorrect tries
Brute-force login (combined defenses)	CAPTCHA + rate limiting enabled	Completely blocked – No password guessed
Login form with CAPTCHA	One correct login with CAPTCHA enabled	Success – Human login allowed
CSRF attack (no protection)	No CSRF token, SameSite not set	Success – Email changed via attack page
CSRF attack (with CSRF token)	Token verification enabled using Flask-WTF	Blocked – Missing/invalid CSRF token
CSRF attack (with SameSite cookie)	SameSite set to ‘Strict’ on session cookie	Blocked – Browser refused cross-site request
Combined defense for CSRF	CSRF token + SameSite cookie applied	Attack failed – Email unchanged

Detailed results of tests before and after applying layered defenses.

7. Conclusion

This project demonstrates the importance of combining multiple layers of defense in web applications. Strong protection is provided by methods like CAPTCHA, rate restriction, CSRF tokens, and SameSite cookies.

The solution adheres to fundamental security concepts, including securing by default, failing securely, and minimizing the attack surface.

8. Future

- Add logging and alerting for brute-force and CSRF attempts.
- Visualize attack attempts using data charts.
- Extend CSRF protection to AJAX and REST API endpoints.
- Deploy the app securely using HTTPS and a production WSGI server.

This project simulates how layered defenses might successfully reduce common threats, reinforcing practical knowledge of web security.